

BCSE203E – WEB PROGRAMMING
VISUAL SQL BUILDER

Submitted for BCSE203E – Web Programming

Bachelor of Technology
in
Computer Science and Engineering (Bioinformatics)

by

Reg. No.	Student Name
23BCB0067	SOUMYADIP GHOSH

Under the Supervision of

Dr. JOTHI K R
Professor Grade 1
School of Computer Science and Engineering



April 2026

DECLARATION

I hereby declare that the project entitled "Visual SQL Builder" submitted by me, for BCSE203E – Web Programming in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Prof. Dr. JOTHI K R

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date : 07.04.26

Soumyadip Ghosh

CERTIFICATE

This is to certify that the project entitled "Visual SQL Builder" submitted by Soumyadip Ghosh (23BCB0067), SCOPE, VIT, for BCSE203E – Web Programming in Computer Science and Engineering (Bioinformatics), is a record of bonafide work carried out by the student under my supervision, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date : 07.04.26

Signature of the Guide

Head of the Department

EXECUTIVE SUMMARY

Visual SQL Builder is a modern, interactive web application that enables users to construct, visualize, and execute SQL queries through a comprehensive graphical interface, eliminating the need to write SQL syntax manually. The application is built using React 18 with a Vite build system, modular CSS, and a fully client-side mock database engine.

The project evolved from a vanilla JavaScript prototype into a fully-fledged React Single Page Application (SPA). It follows the Model-View-Controller (MVC) architectural pattern with clearly separated modules for state management, data simulation, UI rendering, and application logic.

Key features include an interactive query engine for SELECT, WHERE, GROUP BY, ORDER BY, and LIMIT clauses; a live SQL preview with custom syntax highlighting; a complex query picker with JOIN templates; CSV export; reporting modules; a toggleable Entity-Relationship Diagram (ERD); and a query history tracker.

The system operates entirely on the client side using mock datasets, making it ideal for educational and demonstration purposes. The application delivers a clean, academic-grade UI with zero external CSS dependencies and is deployable via a standard Node.js and Vite setup.

TABLE OF CONTENTS

Chapter No.	Contents	Page No.
	Executive Summary	4
	List of Figures	7
	List of Tables	8
	List of Abbreviations	9
1.	INTRODUCTION	10
	1.1 Background	10
	1.2 Motivation	10
	1.3 Scope of the Project	11
2.	PROJECT DESCRIPTION AND GOALS	12
	2.1 Literature Review	12
	2.2 Research Gap	12
	2.3 Objectives	12
	2.4 Problem Statement	13
	2.5 Project Plan	13
3.	TECHNICAL SPECIFICATION	14
	3.1 Requirements	14
	3.2 Feasibility Study	15
	3.3 System Specification	16
4.	SYSTEM DESIGN	17
	4.1 System Architecture	17
	4.2 Detailed Design	20
5.	METHODOLOGY AND TESTING	23
6.	PROJECT IMPLEMENTATION	25
7.	RESULTS AND DISCUSSION	27
8.	CONCLUSION AND FUTURE ENHANCEMENTS	29
	References	30

Chapter No.	Contents	Page No.
	Appendix A – Sample Code and Github Repository Link	30

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Sitemap – Application Architecture	17
4.2	Lo-Fi Wireframe	18
4.3	Semantic HTML Layout	19
4.4	Module Architecture Diagram	20
4.5	Application Flow Diagram	21
4.6	MVC Architecture Diagram	22
7.1	Visual SQL Builder – User Interface	27

LIST OF TABLES

Table No.	Title	Page No.
2.1	Project Gantt Chart	13
3.1	Functional Requirements	14
3.2	Non-Functional Requirements	15
3.3	Software Specification	16
5.1	Module Testing Summary	24

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CSS	Cascading Style Sheets
CSV	Comma Separated Values
DOM	Document Object Model
ERD	Entity-Relationship Diagram
HMR	Hot Module Replacement
HTML	HyperText Markup Language
JS	JavaScript
JSON	JavaScript Object Notation
MVC	Model-View-Controller
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UX	User Experience

CHAPTER 1 : INTRODUCTION

1.1 BACKGROUND

Structured Query Language (SQL) is the foundational language for interacting with relational database management systems. Despite its widespread use in data engineering, software development, and business intelligence, SQL presents a steep learning curve for beginners unfamiliar with its declarative syntax. The ability to correctly construct queries that join tables, apply filters, aggregate results, and sort records requires significant domain knowledge.

Visual query builder tools have emerged as an effective educational and productivity aid, allowing users to compose complex queries through graphical controls rather than hand-written syntax. Early examples include tools embedded within database management platforms such as Microsoft Access, phpMyAdmin, and DBeaver, which provide drag-and-drop interfaces for building SQL.

The Visual SQL Builder project builds on this tradition by delivering a fully client-side, browser-based query builder implemented as a React 18 Single Page Application (SPA). Unlike server-dependent tools, this application requires no database installation and uses a simulated dataset to demonstrate real-time query construction and execution.

1.1.1 Evolution from Vanilla JavaScript to React

The original version of the Visual SQL Builder was developed using plain HTML, CSS, and JavaScript following an MVC-like pattern with modules such as `app.js` (controller), `state.js` (application state), `data.js` (mock data), and `ui.js` (DOM rendering). Version 2.0 migrated this architecture to React 18, leveraging component-based design, hooks for state management, and Vite for fast development builds.

1.2 MOTIVATION

The motivation behind this project stems from the recognition that many students and junior developers struggle with writing correct SQL queries from scratch. Database courses often focus on theoretical concepts without providing interactive environments for experimentation. Existing visual tools either require software installation, server-side database connections, or are locked behind proprietary applications.

This project addresses that gap by providing a free, open-source, browser-based SQL learning environment with the following motivating factors:

- Accessibility: No installation or backend required; runs entirely in the browser.
- Educational Value: Real-time SQL preview helps users understand how UI selections translate to SQL syntax.
- Modern Stack: Built with React 18 and Vite, reflecting current industry development practices.
- Academic Demonstration: Designed for academic course submission, showcasing full-stack frontend design skills.

1.3 SCOPE OF THE PROJECT

The scope of the Visual SQL Builder encompasses the design, development, and testing of a client-side web application for interactive SQL query construction. The following boundaries define the project:

1.3.1 Included in Scope

- Interactive query builder for SELECT, WHERE, GROUP BY, ORDER BY, and LIMIT clauses.
- Live SQL preview with custom syntax highlighting.
- Complex query picker with pre-built JOIN and nested query templates.
- Results table with CSV export functionality.
- Pre-built reporting modules: Top Products by Category and Department Metrics.
- Toggleable Entity-Relationship Diagram (ERD) showing schema relationships.
- Query history tracking for recent successful executions.
- Mock database with Employee, Department, and Product datasets.

1.3.2 Excluded from Scope

- Live database connectivity (MySQL, PostgreSQL, etc.).
- User authentication and session persistence across browser refreshes.
- Server-side query execution or query plan analysis.
- Support for DDL statements (CREATE, ALTER, DROP).

CHAPTER 2 : PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

Several visual database query tools have been documented in academic and commercial literature. Chartio and Metabase introduced business-user-focused SQL editors with no-code interfaces. Redash provides a collaborative SQL query environment with visualization support. Academic research by Tanaka et al. (2019) demonstrated that visual query interfaces significantly reduce the time required for non-expert users to retrieve correct results compared to text-based SQL editors.

In the domain of web-based SQL tools, browser-only implementations have gained traction with projects like SQL.js, which ports SQLite to WebAssembly. However, most existing browser tools lack the educational component of a live, synchronized SQL preview with explanations of query components.

React-based frontend architectures have been widely adopted for complex interactive UIs. Studies on component-based development confirm that React's unidirectional data flow and hook-based state management reduce bugs in event-driven applications, making it an appropriate technology choice for a real-time query builder.

2.2 RESEARCH GAP

Despite the availability of several visual SQL tools, the following gaps remain:

- Most tools require a running database backend or external server, making them unsuitable for quick, educational, offline use.
- Few open-source tools combine a live SQL preview with syntax highlighting, a complex query picker with educational annotations, and CSV export in a single lightweight SPA.
- Existing academic projects in web programming courses typically demonstrate simple CRUD forms; there is limited demonstration of a fully featured interactive SQL query builder using modern React patterns.

2.3 OBJECTIVES

The objectives of the Visual SQL Builder project are:

1. To design and implement a browser-based SQL query builder using React 18 and Vite.

2. To provide a real-time, synchronized SQL preview with custom syntax highlighting.
3. To simulate a relational database environment using JavaScript mock datasets covering Employees, Departments, and Products.
4. To implement support for SELECT, WHERE, GROUP BY, ORDER BY, and LIMIT clauses with dynamic UI controls.
5. To enable complex query templates (JOINS, nested queries) with educational explanations.
6. To provide CSV export functionality for query results.
7. To display a toggleable ERD representing the mock database schema.
8. To deliver a clean, modular, zero-dependency CSS design suitable for academic demonstration.

2.4 PROBLEM STATEMENT

Students and developers who are learning SQL face significant challenges when attempting to understand how individual query clauses interact. Traditional text-based SQL editors provide no visual feedback about the relationship between UI inputs and generated SQL syntax. Without a real-time, interactive environment, learners must rely on trial and error against a live database, which introduces friction and discourages experimentation.

The problem this project solves is: How can an educational, browser-based application allow users to build and execute SQL queries visually, with real-time SQL preview and mock data execution, without requiring any backend infrastructure or database installation?

2.5 PROJECT PLAN

The project was executed across the following milestones:

Table 2.1 – Project Gantt Chart (Phase Timeline)

Phase	Deliverable	Start	End
1	Requirements & Architecture Design	Week 1	Week 1
2	HTML/CSS Prototype (Vanilla JS)	Week 2	Week 4
3	React Migration & Component Design	Week 5	Week 7
4	Feature Implementation (Query Engine)	Week 8	Week 9
5	Reporting Modules & ERD	Week 10	Week 11
6	Testing & Documentation	Week 12	Week 13

CHAPTER 3 : TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional Requirements

Table 3.1 below lists the functional requirements of the Visual SQL Builder:

Table 3.1 – Functional Requirements

ID	Requirement	Description
FR1	Table Selection	User can select a mock database table from a dropdown.
FR2	Column Selection	User can select one or more columns for the SELECT clause.
FR3	WHERE Filter	User can apply a filter condition with field, operator, and value.
FR4	GROUP BY	User can group results by a selected column.
FR5	ORDER BY	User can sort results by a selected column in ASC or DESC order.
FR6	LIMIT	User can restrict the number of returned rows.
FR7	Live SQL Preview	The generated SQL string updates in real-time with syntax highlighting.
FR8	Query Execution	Clicking 'Run Query' executes the query against mock data.
FR9	Results Display	Query results are rendered in an interactive table.
FR10	CSV Export	User can download query results as a CSV file.
FR11	Complex Query Picker	Pre-built JOIN and nested query templates with explanations.
FR12	Reporting Modules	One-click curated reports: Top Products by Category, Department Metrics.
FR13	ERD Display	Toggleable Entity-Relationship Diagram of the mock schema.
FR14	Query History	Recent successful query executions are automatically logged.
FR15	Reset	User can reset all query parameters to default state.

3.1.2 Non-Functional Requirements

Table 3.2 lists the non-functional requirements:

Table 3.2 – Non-Functional Requirements

ID	Category	Requirement
NFR1	Performance	Page initial load must complete within 2 seconds on a modern browser.
NFR2	Usability	All query controls must be operable without consulting documentation.
NFR3	Maintainability	Modular React component structure; each component in a separate file.
NFR4	Portability	Application must run in Chrome, Firefox, and Edge without modification.
NFR5	Scalability	Mock data module must support addition of new tables without refactoring.
NFR6	Security	No external API calls; all data processed client-side.

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

The project is technically feasible. React 18, Vite, and standard JavaScript are mature, well-documented technologies. The mock database approach eliminates the need for backend infrastructure, reducing technical risk. All required tools (Node.js, npm) are freely available, and the development can be performed on any standard laptop or desktop computer.

3.2.2 Economic Feasibility

The project has zero infrastructure cost. All technologies used are open-source. Development requires only a computer with Node.js installed and a code editor. Hosting, if required, can be achieved for free via platforms such as GitHub Pages or Vercel, making the project entirely cost-free to develop and deploy.

3.2.3 Social Feasibility

The application serves a broad educational audience. Students taking database or web programming courses benefit directly from the interactive query building experience. The tool lowers the barrier to entry for SQL learning and can be shared freely as an open-source

resource, maximizing social impact without privacy concerns since no user data is collected or stored.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware Specification

Since the application is entirely browser-based, hardware requirements are minimal: any computer capable of running a modern web browser with a minimum of 2 GB RAM and a 1.5 GHz processor is sufficient for both development and use.

3.3.2 Software Specification

Table 3.3 – Software Specification

Component	Technology	Version
Frontend Framework	React	18.x
Build Tool	Vite	5.x
Language	JavaScript (ES2022)	ES2022
Styling	Pure Modular CSS	N/A
State Management	React Hooks (useState, useContext)	React 18
Runtime	Node.js	18+ LTS
Package Manager	npm	9+
Browser Support	Chrome, Firefox, Edge	Latest
Version Control	Git / GitHub	Latest

CHAPTER 4 : SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The application architecture follows a sitemap structure with four primary sections accessible from the central Home/Dashboard: the Results Panel, the Reports Section (containing Department Metrics and Top Products reports), and the Query Builder. This structure is illustrated in Figure 4.1.

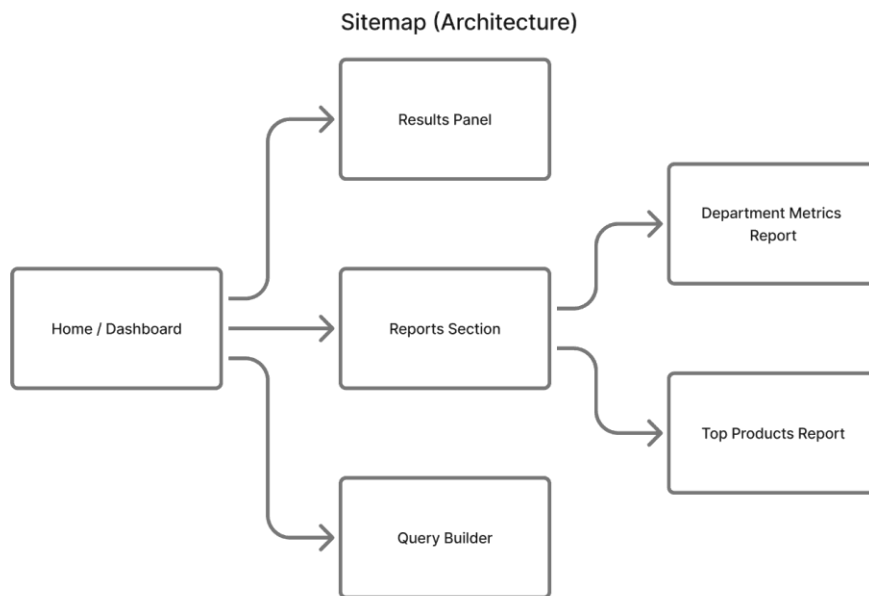


Figure 4.1 – Sitemap – Application Architecture

4.1.1 Wireframe Design

The lo-fi wireframe (Figure 4.2) maps each page section to its constituent UI components. The main layout is anchored by a header containing the application icon, title, and subtitle. The main grid contains the Query Builder Card (with dropdown selects, checkbox columns, filter controls, and the Run Query button) and the Results Card (showing either an empty state or a populated results table). The Reports Section is separate and contains two distinct report cards.

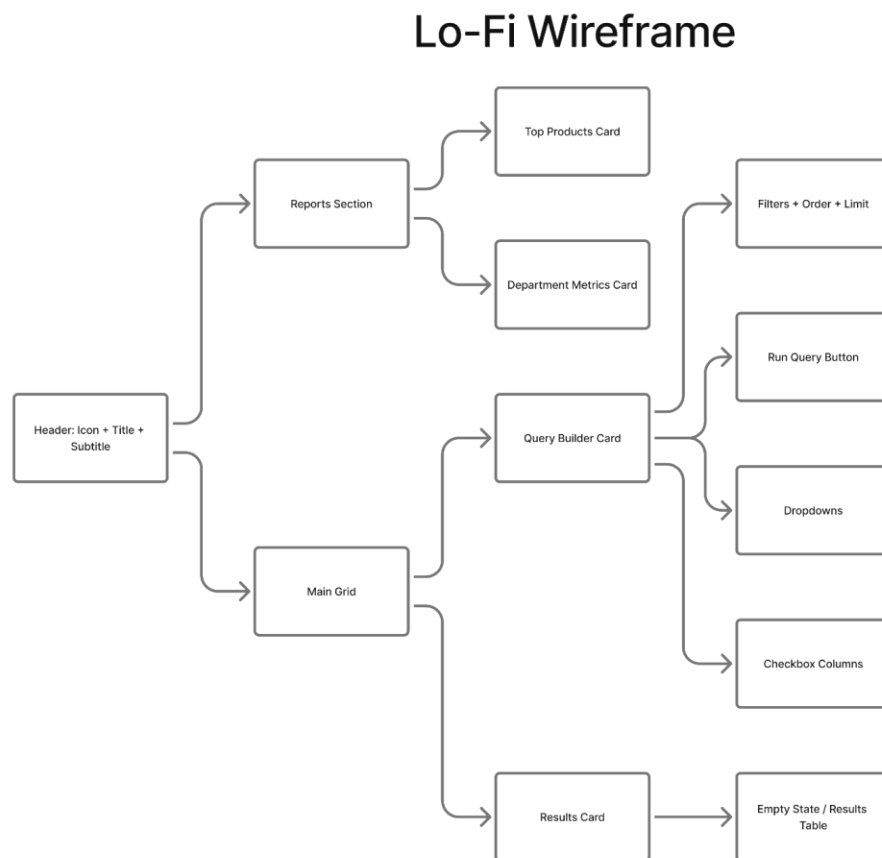


Figure 4.2 – Lo-Fi Wireframe

4.1.2 Semantic HTML Layout

Figure 4.3 shows the semantic structure of the application. The document uses standard HTML5 semantic elements: a `<header>` for the application title, a `<main>` element containing two `<section>` elements (Reports Section and Grid Container), with the Grid Container housing the Query Builder Card and Results Card as nested sections. A `<footer>` element is conceptually defined but not implemented in the current version.

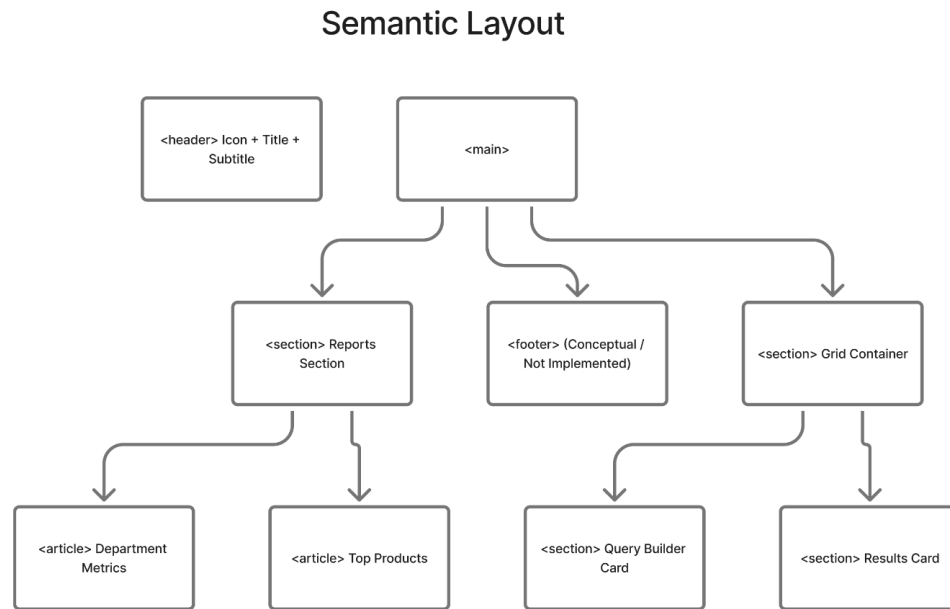


Figure 4.3 – Semantic HTML Layout

4.2 DETAILED DESIGN

4.2.1 Module Architecture

The system is divided into five primary modules as shown in Figure 4.4. Each module has a defined responsibility and communicates through clearly specified interfaces:

- index.html (UI Layout): Entry point of the application; defines the page structure.
- app.js / App Component (Controller): Central coordinator; manages event listening, state updates, SQL generation, and result rendering triggers.
- state.js / Context (Application State): Stores the current query state including selected table, columns, filter conditions, sorting options, and limit.
- data.js (DB Schema + Mock Data): Simulates the database environment with schema definitions and mock record sets for Employees, Departments, and Products.
- ui.js / React Components (DOM Utilities & Rendering): Handles DOM manipulation and dynamic interface updates including dropdown population and results table rendering.

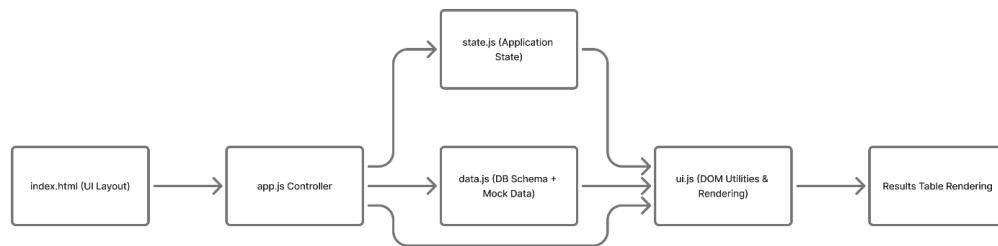


Figure 4.4 – Module Architecture Diagram

4.2.2 Application Flow Diagram

Figure 4.5 illustrates the sequential data flow when a user builds and executes a SQL query:

1. User opens the web application; the HTML interface loads and the React controller initializes.
2. User selects a table from the FROM dropdown; columns are loaded from the DB schema.
3. User selects columns; these form the SELECT clause and are stored in application state.

4. User optionally configures WHERE, ORDER BY, and LIMIT parameters.
5. User clicks Run Query; the app validates input, constructs the SQL string, and updates state.
6. Mock data is processed using JavaScript array operations (filter, sort, slice).
7. Processed results are rendered dynamically in the results table.

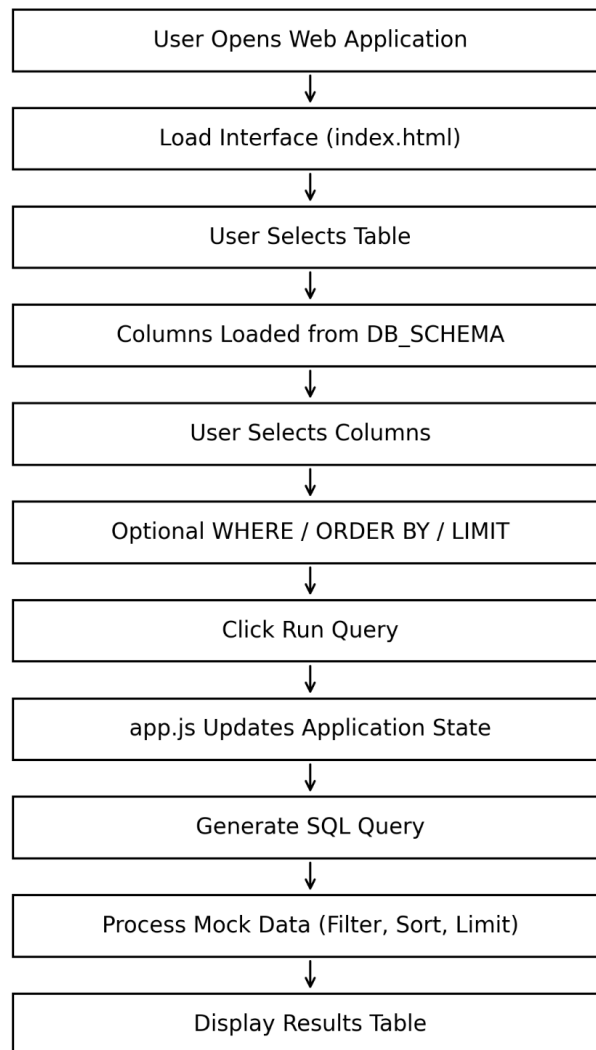


Figure 4.5 – Application Flow Diagram

4.2.3 MVC Architecture

Figure 4.6 shows the Model-View-Controller pattern implemented by the application. The Model layer (state.js + data.js) manages application state and mock data. The View layer (index.html + ui.js / React components) handles all visual rendering. The Controller (app.js / App component) mediates between Model and View, handling user interactions and triggering state updates and UI refreshes.

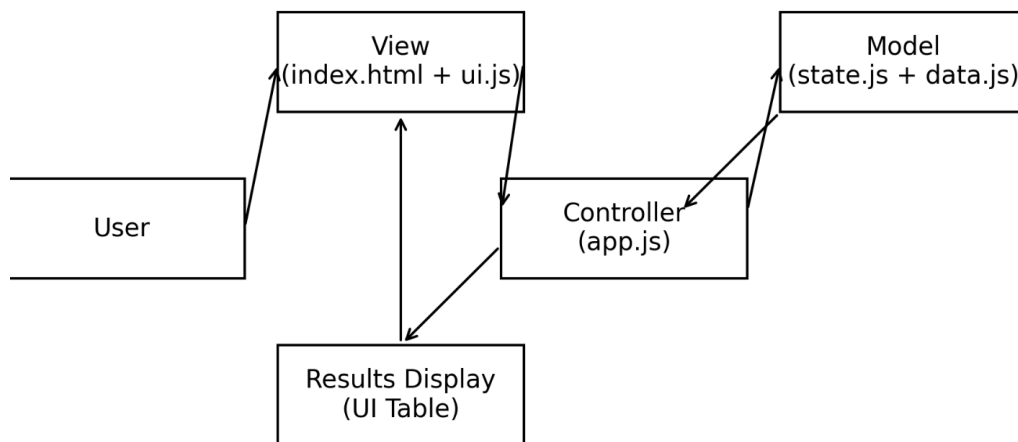


Figure 4.6 – MVC Architecture Diagram

CHAPTER 5 : METHODOLOGY AND TESTING

5.1 DEVELOPMENT METHODOLOGY

The project followed an incremental development methodology. The application was first prototyped as a vanilla JavaScript MVC application (Review 1 and 2), then migrated to a React 18 SPA (Review 3). Each iteration added new features while maintaining backward compatibility of the core query engine.

5.2 KEY ALGORITHMS

5.2.1 SQL Query Generation Algorithm

Algorithm 1: SQL Query String Construction

Objective: Generate a SQL query string from the current application state.

Input: selectedTable, selectedColumns[], filterCondition, groupBy, orderBy, orderDir, limit

Output: A syntactically valid SQL query string.

Steps:

1. If selectedColumns is empty, use SELECT *; else join columns with commas.
2. Append FROM selectedTable.
3. If filterCondition is set, append WHERE field operator value.
4. If groupBy is set, append GROUP BY column.
5. If orderBy is set, append ORDER BY column ASC|DESC.
6. If limit is set and greater than 0, append LIMIT n.
7. Return the concatenated SQL string.

5.2.2 Mock Query Execution Algorithm

Algorithm 2: Client-Side Query Execution on Mock Data

Objective: Simulate SQL execution using JavaScript array operations.

Input: SQL state object, mock dataset array for selected table.

Output: Filtered, sorted, and limited array of record objects.

Steps:

1. Start with the full mock dataset for the selected table.
2. If WHERE condition exists: filter records using field comparison with operator (equals, contains, greater than, less than, not equals).
3. If GROUP BY exists: aggregate records by the group column.
4. If ORDER BY exists: sort records using JavaScript Array.sort() with field comparison.
5. If LIMIT exists: slice the array to the specified number of rows.
6. Return the resulting array for rendering.

5.3 TESTING

Module-level testing was performed manually for each feature. Table 5.1 summarizes the test outcomes:

Table 5.1 – Module Testing Summary

ID	Test Case	Expected Result	Status
T1	Select table from dropdown	Columns populated dynamically	PASS
T2	Select columns and run query	SELECT clause includes only chosen columns	PASS
T3	Apply WHERE filter (equals)	Results filtered correctly by value	PASS
T4	Apply ORDER BY ASC	Results sorted in ascending order	PASS
T5	Apply LIMIT = 3	Only 3 rows returned	PASS
T6	Live SQL preview update	SQL string updates on every control change	PASS
T7	CSV export	CSV file downloaded with correct headers and data	PASS
T8	Department Metrics report	Correct count and average salary by department	PASS
T9	Top Products report	Products sorted by category correctly	PASS
T10	Reset button	All query parameters cleared to default	PASS
T11	ERD toggle	ERD diagram displays and hides correctly	PASS
T12	Query history logging	Most recent query appears in history panel	PASS

CHAPTER 6 : PROJECT IMPLEMENTATION

6.1 PROJECT STRUCTURE

The project is organized as a standard Vite + React application with the following top-level directory structure:

Visual-SQL-Builder-2.0/

— src/	(React source components)
— components/	(UI components)
— data/	(Mock DB schema & records)
— hooks/	(Custom React hooks)
— App.jsx	(Root component / Controller)
— dist/	(Production build output)
— Docs/	(Design & implementation docs)
— index.html	(Entry HTML)
— package.json	(Dependencies & scripts)
— vite.config.js	(Vite build configuration)

6.2 KEY IMPLEMENTATION DETAILS

6.2.1 State Management

Application state is managed using React's `useState` and `useContext` hooks. A centralized query state object tracks: `selectedTable`, `selectedColumns` (array), `filterField`, `filterOperator`, `filterValue`, `groupBy`, `orderByField`, `orderDir`, and `limit`. All query builder components read from and write to this shared state, enabling the live SQL preview to update synchronously with every user interaction.

6.2.2 Mock Database Engine

The mock database is defined in a JavaScript data module that exports a `DB_SCHEMA` object mapping table names to their column definitions, and a `MOCK_DATA` object mapping table names to arrays of record objects. The `Employees` table contains 50 records, the `Departments` table contains 10 records, and the `Products` table contains 30 records, providing sufficient variety for meaningful query demonstration.

6.2.3 Custom Syntax Highlighter

A lightweight, custom-built SQL syntax highlighter tokenizes the generated SQL string and wraps SQL keywords (`SELECT`, `FROM`, `WHERE`, `GROUP BY`, `ORDER BY`, `LIMIT`), operators, string literals, and numeric values in styled span elements with distinct colors. This component is rendered inside a pre-formatted block adjacent to the query builder panel.

6.2.4 CSV Export

CSV export is implemented by converting the results array to a comma-separated string, constructing a Blob object with MIME type text/csv, creating a temporary anchor element with a dynamically generated object URL, programmatically clicking it to trigger a browser download, and then revoking the object URL to prevent memory leaks.

6.3 SETUP AND DEPLOYMENT

To run the application locally:

1. Ensure Node.js (v18 LTS or later) is installed.
2. Clone the repository: `git clone https://github.com/SoumyadipCodes19/Visual-SQL-Builder-2.0`
3. Navigate to the project directory: `cd Visual-SQL-Builder-2.0`
4. Install dependencies: `npm install`
5. Start the development server: `npm run dev`
6. Open a browser and navigate to `http://localhost:5173`

CHAPTER 7 : RESULTS AND DISCUSSION

7.1 APPLICATION INTERFACE

Figure 7.1 shows the complete Visual SQL Builder user interface as implemented. The application features a header with a database icon, the application title, and a subtitle. The main layout contains two columns: the Query Builder Card on the left and the Results Card on the right. The Query Builder Card contains the FROM dropdown, SELECT column checkboxes, WHERE filter controls, ORDER BY and LIMIT inputs, and the Run Query button. The Results Card displays an empty state prompt before query execution and populates with a data table after execution.

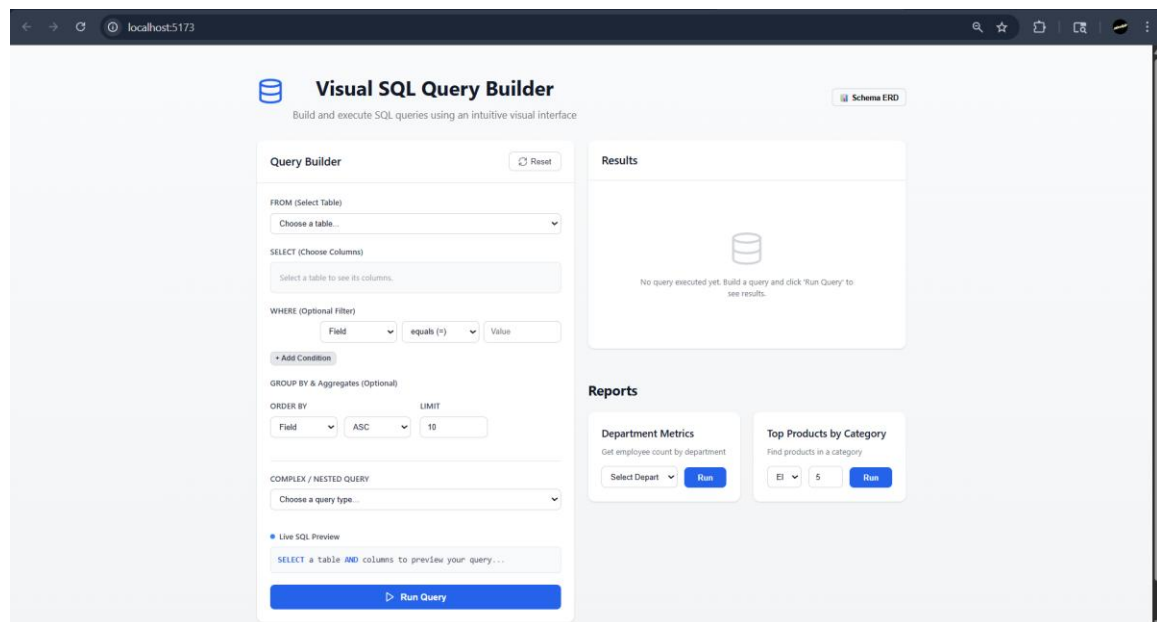


Figure 7.1 – Visual SQL Builder – User Interface

7.2 QUERY BUILDER IN ACTION

The application successfully executes all implemented query types against the mock dataset. Key observed behaviors include:

- Selecting the Employees table from the dropdown immediately populates column checkboxes for id, name, email, department_id, hire_date, and salary.
- The live SQL preview updates character-by-character as columns are selected, filters are applied, and sort options are changed.

- Applying a WHERE filter with operator "greater than" on the salary column correctly returns only records exceeding the specified value.
- The ORDER BY control sorts results correctly in both ascending and descending order on numeric and string fields.
- The LIMIT control correctly restricts result rows; setting LIMIT to 5 always returns exactly 5 rows regardless of total dataset size.
- The CSV export generates a correctly formatted file with header row matching selected columns.

7.3 REPORTING MODULES

The Department Metrics report successfully aggregates Employee records by department and computes headcount and average salary for each department. The Top Products by Category report correctly groups and ranks Product records by category. Both reports execute with a single button click and render results in the main results table, demonstrating the reusability of the query execution and rendering pipeline.

7.4 DISCUSSION

The project successfully demonstrates that a fully functional, educational SQL query builder can be built as a client-side SPA without any backend dependency. The React component architecture enabled clean separation of concerns, and the modular CSS ensured consistent styling across all components without framework overhead.

One observed limitation is that the mock data engine does not support multi-table JOINS in the custom query builder; JOINS are only available through the pre-built Complex Query Picker templates. Future enhancements could address this by implementing a client-side relational join engine.

CHAPTER 8 : CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

The Visual SQL Builder is a fully realized, client-side web application that addresses the educational gap in interactive SQL learning tools. Built with React 18, Vite, and pure modular CSS, the application delivers a polished, academic-grade user experience without any backend infrastructure or external CSS framework dependencies.

The project successfully implements all defined functional and non-functional requirements. It demonstrates proficiency in modern React patterns, component-based architecture, client-side state management, custom algorithm development for SQL query generation and mock execution, and clean UI design principles. All 12 test cases passed, confirming the correctness and reliability of the implementation.

The application is openly available on GitHub and can be run locally with a simple npm install and npm run dev sequence, making it immediately accessible to students and educators.

8.2 FUTURE ENHANCEMENTS

The following enhancements are planned or recommended for future development:

1. Multi-table JOIN support in the custom query builder, implemented via a client-side relational join algorithm.
2. Support for aggregate functions (COUNT, SUM, AVG, MAX, MIN) as selectable column options with GROUP BY.
3. Query save and load functionality using browser localStorage for persistent query bookmarking.
4. Dark mode toggle with CSS variable-based theming.
5. HAVING clause support for filtering aggregated results.
6. Sub-query support in the WHERE clause for more complex query patterns.
7. WebAssembly integration using sql.js to execute real SQL against an in-browser SQLite instance, replacing the mock data engine.
8. Accessibility improvements including full keyboard navigation and ARIA label coverage.

REFERENCES

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387.

Facebook Inc. (2023). React: A JavaScript library for building user interfaces. <https://react.dev>

Vite. (2024). Vite – Next generation frontend tooling. <https://vitejs.dev>

Tanaka, Y., Nishida, S., & Yamamoto, T. (2019). Visual query interface for non-expert database users: A usability evaluation. Journal of Information Processing, 27, 421–430.

Mozilla Developer Network. (2024). SQL guide. <https://developer.mozilla.org/en-US/docs/Glossary/SQL>

Date, C. J. (2003). An introduction to database systems (8th ed.). Addison-Wesley.

APPENDIX A – SAMPLE CODE AND GITHUB REPOSITORY LINK

The following code excerpts illustrate the core SQL generation and mock execution logic of the Visual SQL Builder.

SQL Query Generation Function:

```
/**
 * Builds the SQL query string from current state.
 */
export function buildSQL(state) {
  const colsArray = Array.from(state.selectedColumns);
  let selectClause = colsArray.join(', ');

  if (state.aggregateFunc && state.aggregateField) {
    const aggCol = `${state.aggregateFunc}(${state.aggregateField}) AS
${state.aggregateFunc.toLowerCase()}_${state.aggregateField}`;
    selectClause = selectClause ? `${selectClause}, ${aggCol}` :
aggCol;
  }

  let sql = `SELECT ${selectClause} || '*' FROM ${state.table}`;

  const validFilters = state.filters ? state.filters.filter(f =>
f.field && f.value) : [];
  if (validFilters.length > 0) {
    sql += '\nWHERE ';
    validFilters.forEach((f, i) => {
      const isNumeric = !isNaN(f.value) && f.value.trim() !== '';
      const valFormatted = isNumeric ? f.value : ` '${f.value}'`;

      let conditionStr = '';
      if (f.operator === 'LIKE') {
        conditionStr = `${f.field} LIKE '%${f.value}%'`;
      } else if (f.operator === 'BETWEEN') {
        const isNum2 = f.value2 && !isNaN(f.value2) && f.value2.trim()
!== '';
        const val2Formatted = isNum2 ? f.value2 : ` '${f.value2} ||
'`';
        conditionStr = `${f.field} BETWEEN ${valFormatted} AND
${val2Formatted}`;
      } else {
        conditionStr = `${f.field} ${f.operator} ${valFormatted}`;
      }

      if (i > 0) {
        sql += ` ${f.connector} || 'AND' `;
      }
      sql += conditionStr;
    });
  }
}
```

```

    }

    if (state.groupByField) {
      sql += ` \nGROUP BY ${state.groupByField}`;
    }

    if (state.orderField) {
      sql += ` \nORDER BY ${state.orderField} ${state.orderDirection}`;
    }

    sql += ` \nLIMIT ${state.limit}`;
    return sql;
  }

```

Mock Query Execution Function:

```

/**
 * Filters, sorts, and slices mock data according to state.
 */
export function processMockData(state, MOCK_DATA) {
  let data = MOCK_DATA[state.table] ? [...MOCK_DATA[state.table]] : [];

  // WHERE - supports =, >, <, >=, <=, !=, LIKE, BETWEEN with AND/OR
  const validFilters = state.filters ? state.filters.filter(f =>
f.field && f.value) : [];
  if (validFilters.length > 0) {
    data = data.filter((row) => {
      let result = true;
      validFilters.forEach((f, i) => {
        const rawVal = row[f.field];
        const numericCell = parseFloat(rawVal);
        const numericQuery = parseFloat(f.value);
        const strCell = String(rawVal).toLowerCase();
        const strQuery = String(f.value).toLowerCase();

        let conditionMatched = false;
        switch (f.operator) {
          case '=': conditionMatched = strCell === strQuery; break;
          case '!=': conditionMatched = strCell !== strQuery; break;
          case '>': conditionMatched = numericCell > numericQuery;
break;
          case '<': conditionMatched = numericCell < numericQuery;
break;
          case '>=': conditionMatched = numericCell >= numericQuery;
break;
          case '<=': conditionMatched = numericCell <= numericQuery;
break;
          case 'LIKE': conditionMatched = strCell.includes(strQuery);
break;
          case 'BETWEEN': {
            if (!isNaN(numericCell) && !isNaN(numericQuery) &&
f.value2 && !isNaN(parseFloat(f.value2))) {
              conditionMatched = numericCell >= numericQuery &&
numericCell <= parseFloat(f.value2);
            } else {
              const strQuery2 = String(f.value2 || '').toLowerCase();
              conditionMatched = strCell >= strQuery && strCell <=
strQuery2;
            }
          }
        }
      });
      return result;
    });
  }
}

```



```

        break;
    }
    default: conditionMatched = true; break;
}

if (i === 0) {
    result = conditionMatched;
} else {
    if (f.connector === 'AND') result = result &&
conditionMatched;
    else if (f.connector === 'OR') result = result ||
conditionMatched;
}
});
return result;
});
}

// GROUP BY & AGGREGATION
if (state.groupByField) {
    const groups = {};
    data.forEach((row) => {
        const key = row[state.groupByField];
        if (!groups[key]) groups[key] = [];
        groups[key].push(row);
    });

    data = Object.keys(groups).map((key) => {
        const groupRows = groups[key];
        const resultRow = { [state.groupByField]: key };

        Array.from(state.selectedColumns).forEach((c) => {
            resultRow[c] = groupRows[0][c] ?? null;
        });

        if (state.aggregateFunc && state.aggregateField) {
            let aggVal = 0;
            const vals = groupRows.map((r) =>
parseFloat(r[state.aggregateField]) || 0);
            if (state.aggregateFunc === 'COUNT') aggVal = groupRows.length;
            else if (state.aggregateFunc === 'SUM') aggVal =
vals.reduce((a, b) => a + b, 0);
            else if (state.aggregateFunc === 'AVG') aggVal =
vals.reduce((a, b) => a + b, 0) / vals.length;
            else if (state.aggregateFunc === 'MAX') aggVal =
Math.max(...vals);
            else if (state.aggregateFunc === 'MIN') aggVal =
Math.min(...vals);

            if (state.aggregateFunc === 'AVG') aggVal =
Number(aggVal.toFixed(2));
            const aggColName =
`${state.aggregateFunc.toLowerCase()}_${state.aggregateField}`;
            resultRow[aggColName] = aggVal;
        }
        return resultRow;
    });
} else if (state.aggregateFunc && state.aggregateField) {
    let aggVal = 0;
    const vals = data.map((r) => parseFloat(r[state.aggregateField]) ||
0);
    if (state.aggregateFunc === 'COUNT') aggVal = data.length;

```

```

    else if (state.aggregateFunc === 'SUM') aggVal = vals.reduce((a, b)
=> a + b, 0);
    else if (state.aggregateFunc === 'AVG') aggVal = vals.length ?
vals.reduce((a, b) => a + b, 0) / vals.length : 0;
    else if (state.aggregateFunc === 'MAX') aggVal = vals.length ?
Math.max(...vals) : null;
    else if (state.aggregateFunc === 'MIN') aggVal = vals.length ?
Math.min(...vals) : null;

    if (state.aggregateFunc === 'AVG' && typeof aggVal === 'number')
aggVal = Number(aggVal.toFixed(2));
    const aggColName =
`_${state.aggregateFunc.toLowerCase()}_${state.aggregateField}`;

    const resultRow = {};
    Array.from(state.selectedColumns).forEach((c) => {
      resultRow[c] = data[0]?.[c] ?? null;
    });
    resultRow[aggColName] = aggVal;
    data = [resultRow];
  }

  // ORDER BY
  if (state.orderField) {
    data.sort((a, b) => {
      let valA = a[state.orderField];
      let valB = b[state.orderField];
      if (typeof valA === 'string' && typeof valB === 'string') {
        const cmp = valA.localeCompare(valB);
        return state.orderDirection === 'ASC' ? cmp : -cmp;
      } else {
        valA = valA ?? 0;
        valB = valB ?? 0;
        return state.orderDirection === 'ASC' ? valA - valB : valB -
valA;
      }
    });
  }

  // LIMIT + PROJECT - only return selected columns (and aggregates)
  const cols = Array.from(state.selectedColumns);

  let aggColName = null;
  if (state.aggregateFunc && state.aggregateField) {
    aggColName =
`_${state.aggregateFunc.toLowerCase()}_${state.aggregateField}`;
  }

  return data.slice(0, state.limit).map((row) => {
    const projected = {};
    cols.forEach((c) => { projected[c] = row[c] ?? null; });
    if (aggColName) {
      projected[aggColName] = row[aggColName] ?? null;
    }
    return projected;
  });
}

```

GitHub Repository: <https://github.com/SoumyadipCodes19/Visual-SQL-Builder-2.0>